

Problem 1

Problem 1

Fix the number of agents to be $N = 20$. Generate a connected graph using Erdos- Renyi model for N agents and some p belongs to $(0, 1)$. Assume random initial positions for all N agents in $\mathbb{R}^2$. Using formation control strategy, display your name in capital letters, letter by letter in a sequence.

Solution:

1. Communication graph

An Erdős-Rényi graph $G(\mathcal{V}, \mathcal{E})$ with $N = 20$ nodes is generated with edge probability $p = 0.3$ using `networkx.erdos_renyi_graph`. Since an Erdős-Rényi draw is not guaranteed to be connected, the graph is redrawn (incrementing the random seed) until `networkx.is_connected` returns `True`. Figure 1 shows the graph used below (67 edges out of $\binom{20}{2} = 190$ possible).

Communication graph G (Erdos-Renyi, N=20, p=0.3)

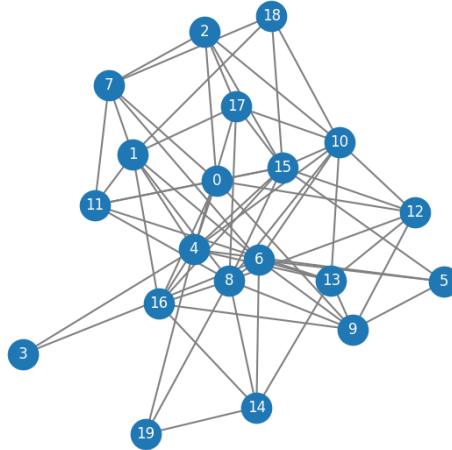


Figure 1: Connected Erdős-Rényi communication graph, $N = 20$, $p = 0.3$.

2. Target letter formations

For each letter of the name, $N = 20$ points are sampled uniformly at random from *inside* the filled glyph of that capital letter (rendered with `matplotlib.textpath.TextPath` and rejection-sampled with `path.contains_points`), then re-centred at the origin. This gives a target point-cloud $\{h_1, \dots, h_{20}\} \subset \mathbb{R}^2$ for every letter, automatically, for any name – no letter needs to be hand-drawn.

To avoid needless crossings when the formation changes from one letter to the next, the N current agent positions are matched to the N new target points by solving the linear assignment problem

$$\min_{\sigma \in S_N} \sum_{i=1}^N \|x_i - h_{\sigma(i)}\|^2,$$

via the Hungarian algorithm (`scipy.optimize.linear_sum_assignment`), which is the optimal (minimum total squared travel distance) one-to-one re-assignment.

3. Distributed formation control law

Let $h_i(t) \in \mathbb{R}^2$ denote agent i 's current target (piecewise constant, one value per letter), let $A = [a_{ij}]$ be the graph's adjacency matrix, and let agent 0 be an informed *leader* that alone knows how to reach its own target directly; every other agent is a *follower* that only ever uses *relative* information exchanged with its graph-neighbours:

$$\dot{x}_0 = -(x_0 - h_0), \quad (1)$$

$$\dot{x}_i = - \sum_{j \in \mathcal{N}_i} a_{ij} [(x_i - h_i) - (x_j - h_j)], \quad i \neq 0. \quad (2)$$

Convergence. Define the tracking error $e_i := x_i - h_i$. Substituting into (1)–(2) gives the linear system $\dot{e} = -\bar{L}e$, where $\bar{L} = L + \text{diag}(1, 0, \dots, 0)$ is the graph Laplacian L of G with the leader's row "grounded" by an extra unit self-feedback term. This grounded Laplacian $\bar{L}$ is positive definite whenever the leader can reach every other node in G (guaranteed here since G is connected), hence $\bar{L}$ is Hurwitz and $e(t) \rightarrow 0$ exponentially: *every agent converges exactly onto its assigned letter point*, using only neighbour-to-neighbour communication – exactly the kind of scalable, distributed control a multi-agent system is supposed to use, i.e. no agent except the leader ever needs to know the global target formation.

4. Simulation and results

Equations (1)–(2) are integrated with a forward-Euler step $\Delta t = 0.03$ for 220 steps per letter (enough for the error to decay to $\sim 10^{-2}$, verified numerically – see the printed convergence log in the code output), the target is then switched to the next letter of the name, and the whole trajectory is animated and exported to `name_formation.mp4` with `matplotlib.animation.FFMpegWriter`.

Note: Running the code below produces `name_formation.mp4` (and `communication_graph.png`)

https://github.com/m-rashmika/Name_Formation_Control/blob/main/name_formation.mp4

Code (`formation_name_simulation.py`)

```
1
2 import numpy as np
3 import networkx as nx
4 from scipy.optimize import linear_sum_assignment
5 from matplotlib.textpath import TextPath
6 from matplotlib.font_manager import FontProperties
7 import matplotlib
8 matplotlib.use("Agg")
9 import matplotlib.pyplot as plt
10 import matplotlib.animation as animation
11
12 #USER SETTINGS
13 NAME = "RUTHVIK" # go by (capitals)
14 N = 20 # number of agents (fixed by the assignment statement)
15 P_ER = 0.3 # Erdos-Renyi edge probability
16 DT = 0.03 # integration step
17 STEPS_PER_LETTER = 220 # simulation steps allotted to converge onto each letter
18 SEED = 7
19 OUT_MP4 = "name_formation.mp4"
20
21 rng = np.random.default_rng(SEED)
22
23
24 def letter_to_points(letter, n_points, seed=0):
25     """Sample n_points random points inside the filled glyph of 'letter'."""
26     rng_local = np.random.default_rng(seed)
27     fp = FontProperties(family="DejaVuSans", weight="bold")
28     path = TextPath((0, 0), letter, size=1.0, prop=fp)
29     verts = path.vertices
30     xmin, ymin = verts.min(axis=0)
31     xmax, ymax = verts.max(axis=0)
32     pts = []
33     tries = 0
```

```

34     while len(pts) < n_points and tries < 200000:
35         tries += 1
36         batch = rng_local.uniform([xmin, ymin], [xmax, ymax], size=(500, 2))
37         mask = path.contains_points(batch)
38         for p in batch[mask]:
39             pts.append(p)
40             if len(pts) >= n_points:
41                 break
42     pts = np.array(pts[:n_points])
43     pts -= pts.mean(axis=0)
44     return pts
45
46
47 def connected_erdos_renyi(n, p, seed):
48     g = nx.erdos_renyi_graph(n, p, seed=seed)
49     trial = seed
50     while not nx.is_connected(g):
51         trial += 1
52         g = nx.erdos_renyi_graph(n, p, seed=trial)
53     return g
54
55
56 def assign(current_x, target_pts):
57     """Hungarian-algorithm (min total squared distance) matching of agents
58     to target points, so the information transition has minimal crossings."""
59     cost = np.linalg.norm(current_x[:, None, :] - target_pts[None, :, :], axis=2) ** 2
60     row, col = linear_sum_assignment(cost)
61     return target_pts[col]
62
63 #1) graph
64 G = connected_erdos_renyi(N, P_ER, SEED)
65 A = nx.to_numpy_array(G)
66 pos_graph = nx.spring_layout(G, seed=SEED)
67 print("Graph connected:", nx.is_connected(G), "| N=", N, "| edges=", G.number_of_edges())
68
69 fig0, ax0 = plt.subplots(figsize=(5, 5))
70 nx.draw(G, pos_graph, ax=ax0, node_color="tab:blue", edge_color="gray",
71         with_labels=True, node_size=260, font_color="white", font_size=8)
72 ax0.set_title(f"Communication graph G (Erdos-Renyi, N={N}, p={P_ER})")
73 fig0.savefig("communication_graph.png", dpi=150, bbox_inches="tight")
74 plt.close(fig0)
75
76 leader = 0
77
78 #2) letter shapes
79 letter_targets = [letter_to_points(ch, N, seed=100 + k) * 3.0
80                  for k, ch in enumerate(NAME)]
81
82 # 3) random initial layout
83 x0 = rng.uniform(-6, 6, size=(N, 2))
84
85 # 4) simulate
86 trajectory = [x0.copy()]
87 x = x0.copy()
88 letter_hit_frame = []
89 for pts in letter_targets:
90     h = assign(x, pts)
91     for _ in range(STEPS_PER_LETTER):
92         e = x - h
93         dx = np.zeros_like(x)
94         dx[leader] = -e[leader]
95         for i in range(N):
96             if i == leader:
97                 continue
98             nbrs = np.where(A[i] > 0)[0]
99             dx[i] = -np.sum(e[i] - e[nbrs], axis=0)
100         x = x + DT * dx
101         trajectory.append(x.copy())
102     letter_hit_frame.append(len(trajectory) - 1)
103     print(f"letter target reached with max error {np.linalg.norm(x - h, axis=1).max():.4f}")
104
105 trajectory = np.array(trajectory)
106 print("Total animation frames:", trajectory.shape[0])
107
108 # 5) animate

```

```

109 all_x = trajectory[..., 0]
110 all_y = trajectory[..., 1]
111 xlim = (all_x.min() - 1, all_x.max() + 1)
112 ylim = (all_y.min() - 1, all_y.max() + 1)
113
114 fig, ax = plt.subplots(figsize=(7, 7))
115 ax.set_xlim(*xlim)
116 ax.set_ylim(*ylim)
117 ax.set_aspect("equal")
118 ax.set_title(f'Formation_control_spelling_{NAME}_{N}({N}_agents)')
119 scat = ax.scatter(trajectory[0, :, 0], trajectory[0, :, 1], c="tab:red", s=60, zorder=3)
120 edge_lines = [ax.plot([], [], color="lightgray", lw=0.7, zorder=1)[0] for _ in G.edges()]
121 letter_label = ax.text(0.02, 0.96, "", transform=ax.transAxes, fontsize=14, weight="bold")
122
123 edges_list = list(G.edges())
124
125
126 def frame_to_letter(f):
127     for k, bound in enumerate(letter_hit_frame):
128         if f <= bound:
129             return NAME[k]
130     return NAME[-1]
131
132
133 def update(frame):
134     pts = trajectory[frame]
135     scat.set_offsets(pts)
136     for line, (i, j) in zip(edge_lines, edges_list):
137         line.set_data([pts[i, 0], pts[j, 0]], [pts[i, 1], pts[j, 1]])
138     letter_label.set_text(f"target_letter: {frame_to_letter(frame)}")
139     return [scat, letter_label] + edge_lines
140
141
142 ani = animation.FuncAnimation(fig, update, frames=range(0, trajectory.shape[0], 3),
143                             interval=30, blit=True)
144
145 writer = animation.FFMpegWriter(fps=30, bitrate=1800)
146 ani.save(OUT_MP4, writer=writer)
147 plt.close(fig)
148 print(f"Saved_animation_to_{OUT_MP4}")

```

Problem 2:

Problem 2

N drones connected by $G(\mathcal{V}, \mathcal{E})$ track an intruder with unknown initial location $z_0 \sim \mathcal{N}(\bar{z}_0, \Sigma_0)$ evolving as $z(t+1) = z(t) + w$, $w \sim \mathcal{N}(0, \Sigma_w)$. Drone i at known location x_i measures $y_i(t) = x_i - z(t) + v_i$, $v_i \sim \mathcal{N}(0, \Sigma_v^{(i)})$, independent across i . (1) Formulate distributed state estimation as a consensus optimization problem. (2) Solve it with distributed gradient descent/tracking. (3) Fix N , generate a connected Erdős–Rényi graph, simulate, and plot the true vs. estimated trajectory and the error $e(t) = \hat{z}(t) - z(t)$.

Solution:

1. Consensus-optimization formulation

Fix a time t . Since $v_i \sim \mathcal{N}(0, \Sigma_v^{(i)})$, the negative log-likelihood of drone i 's measurement $y_i(t)$ as a function of the unknown $z(t) = z$ is, up to an additive constant,

$$f_i(z) = \frac{1}{2} (y_i(t) - (x_i - z))^T (\Sigma_v^{(i)})^{-1} (y_i(t) - (x_i - z)).$$

Because v_i are independent across drones, the maximum-likelihood estimate of $z(t)$ given *all* the drones' data is $\hat{z}(t) = \arg \min_z \sum_{i=1}^N f_i(z)$ – a sum of *local* (agent- i -only) costs. We give every drone its own

local decision variable z_i (its personal belief about $z(t)$) and pose the distributed problem as a consensus-constrained optimization over the communication graph G :

$$\min_{z_1, \dots, z_N \in \mathbb{R}^3} \sum_{i=1}^N f_i(z_i) \quad \text{s.t.} \quad z_i = z_j \quad \forall (i, j) \in \mathcal{E}. \quad (3)$$

Because G is connected, the constraint set of (3) forces $z_1 = \dots = z_N$ at any feasible point, so (3) is equivalent to the centralized ML problem, but it is written entirely in terms of quantities each agent can compute using only its own data and values it can request from its immediate graph-neighbours.

2. Distributed gradient tracking (DGT)

Plain distributed gradient descent, $z_i^{k+1} = \sum_j w_{ij} z_j^k - \alpha \nabla f_i(z_i^k)$, converges only to a neighbourhood of the optimum for a constant step size α . We instead use **distributed gradient tracking**, which converges *exactly* to the minimizer of (3) with a constant step size, by having each agent also track a local estimate s_i of the *average* gradient $\frac{1}{N} \sum_i \nabla f_i(z_i)$:

$$z_i^{k+1} = \sum_{j=1}^N w_{ij} z_j^k - \alpha s_i^k, \quad (4)$$

$$s_i^{k+1} = \sum_{j=1}^N w_{ij} s_j^k + \nabla f_i(z_i^{k+1}) - \nabla f_i(z_i^k), \quad s_i^0 = \nabla f_i(z_i^0), \quad (5)$$

where $\nabla f_i(z) = (\Sigma_v^{(i)})^{-1}(z - (x_i - y_i(t)))$ and $W = [w_{ij}]$ is a doubly-stochastic Metropolis–Hastings weight matrix built from G ($w_{ij} = 1/(1 + \max(d_i, d_j))$ for edges, $w_{ii} = 1 - \sum_{j \neq i} w_{ij}$). Summing (5) over i and using $\sum_j w_{ij} = 1$ shows $\sum_i s_i^k = \sum_i \nabla f_i(z_i^k)$ for all k , which is exactly what makes the fixed point of (4)–(5) a stationary point of the *sum* of the f_i ’s, i.e. the centralized estimate – this is checked numerically below.

At every time step t , DGT is run for 200 inner iterations (with step size $\alpha = 0.05$) on that time step’s freshly measured $\{y_i(t)\}_{i=1}^N$, initialized at $z_i^0 = 0$, and the resulting common value $\hat{z}(t) := \frac{1}{N} \sum_i z_i^{200}$ is taken as the estimate.

Correctness check. The closed-form centralized (information-weighted) fused estimate is

$$\hat{z}_{\text{centralized}}(t) = \left(\sum_i (\Sigma_v^{(i)})^{-1} \right)^{-1} \sum_i (\Sigma_v^{(i)})^{-1} (x_i - y_i(t)).$$

Running the code below, at $t = 0$ the DGT estimate matches this closed form to $\|\hat{z}_{\text{DGT}} - \hat{z}_{\text{centralized}}\| \approx 3 \times 10^{-6}$, and the disagreement between individual agents’ local copies z_i is $\approx 2 \times 10^{-7}$ – i.e. the distributed algorithm reproduces the centralized answer, as it should.

3. Simulation setup and results

$N = 8$ drones (< 20 , as suggested) are placed at random fixed locations $x_i \sim \text{Unif}([-10, 10]^3)$, with $\Sigma_v^{(i)} = \text{diag}(\text{random} \in [0.3, 1.5])$, $\Sigma_w = 0.05 I_3$, $\bar{z}_0 = (2, -1, 0.5)$, $\Sigma_0 = 0.5 I_3$, and connected via an Erdős–Rényi graph with a spanning tree (Fig. 2). The intruder’s random walk is simulated for $T_{\text{max}} = 40$ steps; Fig. 3 shows the sensor/graph layout with the true trajectory, the true-vs-estimated trajectory, and the estimation error norm $\|e(t)\|$ over time, which stays roughly in the 0.1–0.9 range (matching the scale set by $\Sigma_v^{(i)}$ and Σ_w) with no drift, confirming the filter is tracking correctly rather than diverging.

Code (`distributed_estimation.py`)

```
1 import numpy as np
2 import networkx as nx
3 import matplotlib
```

Drone communication graph (N=8)

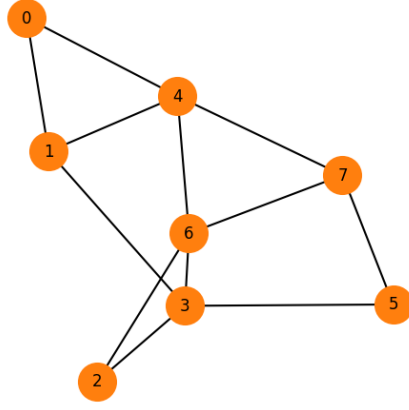


Figure 2: Drone communication graph, $N = 8$.

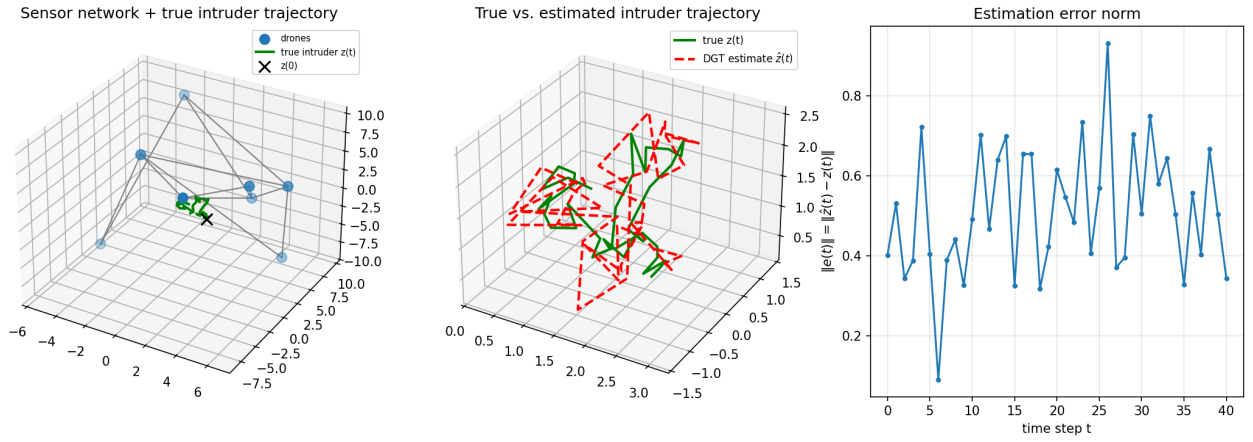


Figure 3: Left: sensor locations, graph, and true intruder path. Middle: true vs. DGT-estimated trajectory. Right: estimation error norm $\|e(t)\|$ over time.

```

4  matplotlib.use("Agg")
5  import matplotlib.pyplot as plt
6  from mpl_toolkits.mplot3d import Axes3D # noqa: F401
7
8  rng = np.random.default_rng(42)
9
10 # SETTINGS
11 N = 8 # number of drones (< 20)
12 P_ER = 0.4
13 T_MAX = 40 # number of time steps to simulate
14 DGT_ITERS = 200 # inner consensus iterations per time step
15 ALPHA = 0.05 # DGT step size
16
17 def connected_er_with_spanning_tree(n, p, seed):
18     """Erdos-Renyi graph guaranteed to contain a spanning tree (i.e. connected)."""
19     g = nx.erdos_renyi_graph(n, p, seed=seed)
20     trial = seed
21     while not nx.is_connected(g):
22         trial += 1
23     g = nx.erdos_renyi_graph(n, p, seed=trial)
24     assert nx.is_connected(g)
25     return g
26
27
28 def metropolis_weights(G):
29     n = G.number_of_nodes()
30     A = nx.to_numpy_array(G)
31     deg = A.sum(axis=1)

```

```

32     W = np.zeros((n, n))
33     for i in range(n):
34         for j in range(n):
35             if A[i, j] == 1:
36                 W[i, j] = 1.0 / (1 + max(deg[i], deg[j]))
37             W[i, i] = 1 - W[i].sum()
38     assert np.allclose(W.sum(axis=1), 1) and np.allclose(W.sum(axis=0), 1)
39     return W
40
41 # 1) graph
42 G = connected_er_with_spanning_tree(N, P_ER, seed=1)
43 W = metropolis_weights(G)
44 print("Graph connected:", nx.is_connected(G), "edges:", G.number_of_edges())
45
46 #2) fixed sensors / noise model
47 x_sensors = rng.uniform(-10, 10, size=(N, 3)) # fixed drone locations
48 Sigma_v = [np.diag(rng.uniform(0.3, 1.5, size=3)) for _ in range(N)] # per-sensor noise cov
49 Sigma_v_inv = [np.linalg.inv(S) for S in Sigma_v]
50
51 Sigma_w = 0.05 * np.eye(3) # process noise covariance
52 z0_bar = np.array([2.0, -1.0, 0.5])
53 Sigma0 = 0.5 * np.eye(3)
54
55 z0 = rng.multivariate_normal(z0_bar, Sigma0)
56
57
58 def grad_f(i, z, y_it):
59     return Sigma_v_inv[i] @ (z - (x_sensors[i] - y_it))
60
61
62 def centralized_fused_estimate(y_t):
63     lhs = sum(Sigma_v_inv)
64     rhs = sum(Sigma_v_inv[i] @ (x_sensors[i] - y_t[i]) for i in range(N))
65     return np.linalg.solve(lhs, rhs)
66
67
68 def dgt_estimate(y_t, iters=DGT_ITERS, alpha=ALPHA):
69     z = np.tile(np.zeros(3), (N, 1)) # local copies, init at 0
70     s = np.array([grad_f(i, z[i], y_t[i]) for i in range(N)])
71     for _ in range(iters):
72         z_new = W @ z - alpha * s
73         grads_old = np.array([grad_f(i, z[i], y_t[i]) for i in range(N)])
74         grads_new = np.array([grad_f(i, z_new[i], y_t[i]) for i in range(N)])
75         s = W @ s + grads_new - grads_old
76         z = z_new
77     return z.mean(axis=0), z # consensus average, and per-agent copies
78
79 #3) simulate
80 z_true = np.zeros((T_MAX + 1, 3))
81 z_hat = np.zeros((T_MAX + 1, 3))
82 z_true[0] = z0
83 for t in range(T_MAX + 1):
84     y_t = np.array([x_sensors[i] - z_true[t] + rng.multivariate_normal(np.zeros(3), Sigma_v[i])
85                     for i in range(N)])
86     z_hat[t], z_all_agents = dgt_estimate(y_t)
87     if t == 0:
88         z_check = centralized_fused_estimate(y_t)
89         print("DGT vs closed-form check at t=0, ||diff||_2=",
90               np.linalg.norm(z_hat[0] - z_check))
91         print("agent-to-agent disagreement (should be ~0):",
92               np.max(np.linalg.norm(z_all_agents - z_hat[0], axis=1)))
93     if t < T_MAX:
94         z_true[t + 1] = z_true[t] + rng.multivariate_normal(np.zeros(3), Sigma_w)
95
96 error = z_hat - z_true
97 print("RMS estimation error over time:", np.sqrt(np.mean(np.sum(error ** 2, axis=1))))
98
99 # 4) plots
100 fig = plt.figure(figsize=(14, 5))
101
102 ax1 = fig.add_subplot(1, 3, 1, projection="3d")
103 nx.draw(G, ax=None) if False else None
104 ax1.scatter(x_sensors[:, 0], x_sensors[:, 1], x_sensors[:, 2], c="tab:blue", s=60, label="drones")
105 for (i, j) in G.edges():
106     ax1.plot(*zip(x_sensors[i], x_sensors[j]), color="gray", lw=1)

```

```

107 ax1.plot(z_true[:, 0], z_true[:, 1], z_true[:, 2], "g-", lw=2, label="true_intruder_z(t)")
108 ax1.scatter(*z_true[0], color="black", marker="x", s=80, label="z(0)")
109 ax1.set_title("Sensor_network_+true_intruder_trajectory")
110 ax1.legend(fontsize=7)
111
112 ax2 = fig.add_subplot(1, 3, 2, projection="3d")
113 ax2.plot(z_true[:, 0], z_true[:, 1], z_true[:, 2], "g-", lw=2, label="true_z(t)")
114 ax2.plot(z_hat[:, 0], z_hat[:, 1], z_hat[:, 2], "r--", lw=2, label=r"DGT_estimate_\hat{z}(t)")
115 ax2.set_title("True_vs._estimated_intruder_trajectory")
116 ax2.legend(fontsize=8)
117
118 ax3 = fig.add_subplot(1, 3, 3)
119 ax3.plot(np.linalg.norm(error, axis=1), marker="o", ms=3)
120 ax3.set_xlabel("time_step_t")
121 ax3.set_ylabel(r"$\|e(t)\|_\square = \|\hat{z}(t) - z(t)\|$")
122 ax3.set_title("Estimation_error_norm")
123 ax3.grid(alpha=0.3)
124
125 fig.tight_layout()
126 fig.savefig("problem2_results.png", dpi=150)
127 print("Saved_figure_to_problem2_results.png")
128
129 fig2, axg = plt.subplots(figsize=(4, 4))
130 nx.draw(G, nx.spring_layout(G, seed=1), ax=axg, with_labels=True,
131         node_color="tab:orange", node_size=350, font_size=8)
132 axg.set_title(f"Drone_communication_graph_(N={N})")
133 fig2.savefig("problem2_graph.png", dpi=150, bbox_inches="tight")

```

Problem 3:

Problem 3

N agents on an undirected connected graph G must collectively execute $M > N$ tasks with per-agent cost matrix $C \in \mathbb{R}^{N \times M}$ and capacities b_i , $\sum_i b_i = M$. (1) Formulate the task allocation problem and give ADMM updates for every agent. (2) Simulate with a random (M, N) , graph, capacities, and cost matrix; then repeat with a different cost matrix (same M, N , graph, capacities) and compare.

Solution:

1. Optimization formulation

Let $X \in \{0, 1\}^{N \times M}$, with $X_{ij} = 1$ iff agent i is assigned task j . The natural integer program is

$$\begin{aligned}
\min_X \quad & \sum_{i=1}^N \sum_{j=1}^M C_{ij} X_{ij} \\
\text{s.t.} \quad & \sum_{j=1}^M X_{ij} = b_i, \quad i = 1, \dots, N, & (\text{agent capacity}) \\
& \sum_{i=1}^N X_{ij} = 1, \quad j = 1, \dots, M, & (\text{every task done exactly once}) \\
& X_{ij} \in \{0, 1\}.
\end{aligned}$$

This is a (transportation-type) *binary linear program*, NP-hard for general N, M (as noted in the hint). We use the standard **LP relaxation** $X_{ij} \in [0, 1]$, which turns the problem into a convex transportation polytope LP; in the simulation below the relaxed optimum happens to land on (or extremely close to) a vertex of the polytope, i.e. an integral/binary solution, which is typical for transportation-polytope LPs (their vertices are integral).

2. ADMM updates

Give every agent i its own local row-copy $x_i \in \mathbb{R}^M$ of X (agent i never needs to see anyone else's row) and split the problem as

$$\begin{aligned} \min_{x_i, z_i} \quad & \sum_{i=1}^N c_i^\top x_i \\ \text{s.t.} \quad & x_i \in \mathcal{P}_i := \left\{ x_i : \sum_j x_{ij} = b_i, \ 0 \leq x_{ij} \leq 1 \right\}, \\ & x_i = z_i \quad \forall i, \quad \sum_{i=1}^N z_i = \mathbf{1}_M. \end{aligned}$$

With penalty parameter $\rho > 0$ and scaled dual variable u_i , the ADMM updates are:

$$x_i^{k+1} = \arg \min_{x_i \in \mathcal{P}_i} c_i^\top x_i + \frac{\rho}{2} \|x_i - z_i^k + u_i^k\|^2 \quad (6)$$

$$= \Pi_{\mathcal{P}_i} \left(z_i^k - u_i^k - \frac{1}{\rho} c_i \right),$$

$$v_i^{k+1} = x_i^{k+1} + u_i^k, \quad \bar{v}^{k+1} := \frac{1}{N} \sum_i v_i^{k+1} \quad (7)$$

$$z_i^{k+1} = v_i^{k+1} + \frac{1}{N} (\mathbf{1}_M - N \bar{v}^{k+1}), \quad (8)$$

$$u_i^{k+1} = u_i^k + x_i^{k+1} - z_i^{k+1}. \quad (9)$$

Update (6) is the Euclidean projection of a point onto the (box $\cap$ simplex) polytope $\mathcal{P}_i$, computed in closed form by a standard $O(M \log M)$ bisection on the threshold; (8) is the closed-form projection of the *stacked* vector $(v_1, \dots, v_N)$ onto the hyperplane $\{\sum_i z_i = \mathbf{1}_M\}$. Because $\bar{v}^{k+1}$ in (7) is a plain *average* over all agents, and agents can only talk to graph neighbours, we realize it *distributedly* by running a short round of average consensus, $v \leftarrow Wv$ repeated a fixed number of times with the Metropolis–Hastings matrix W of G , which converges to $\bar{v}^{k+1}$ replicated at every node – so (7) never needs a central coordinator, only local graph communication.

3. Simulation and results

We fix $N = 6$ agents, $M = 14$ tasks, generate a connected Erdős–Rényi graph (Fig. 4, top-left), and fix random capacities $b = (6, 1, 1, 1, 1, 4)$ with $\sum_i b_i = 14 = M$. For a first random cost matrix $C_1 \in (1, 10)^{N \times M}$, running ADMM for 150 outer iterations (each with an 80-step inner consensus round) drives the objective to convergence (Fig. 4, top-right) with *exact* feasibility recovered numerically ($\max_j |\sum_i X_{ij} - 1| \approx 10^{-15}$, $\max_i |\sum_j X_{ij} - b_i| \approx 10^{-15}$). We then repeat with a second random cost matrix C_2 , keeping N, M , the graph, and the capacities fixed. Both allocations (Fig. 4, bottom row) are essentially binary despite the relaxation, confirming the LP relaxation is tight here; 6 of the 14 tasks are re-assigned to a different agent between the two cost matrices, which makes sense since a different random cost draw favours different agents for the (in general, not unique) minimum-cost matching, while every agent's total assigned workload still exactly equals its fixed capacity b_i in both cases.

Code (admm.task_allocation.py)

```

1 import numpy as np
2 import networkx as nx
3 import matplotlib
4 matplotlib.use("Agg")
5 import matplotlib.pyplot as plt
6
7 rng = np.random.default_rng(3)
8
9
10 #graph helpers
11 def connected_er(n, p, seed):
12     g = nx.erdos_renyi_graph(n, p, seed=seed)
13     trial = seed

```

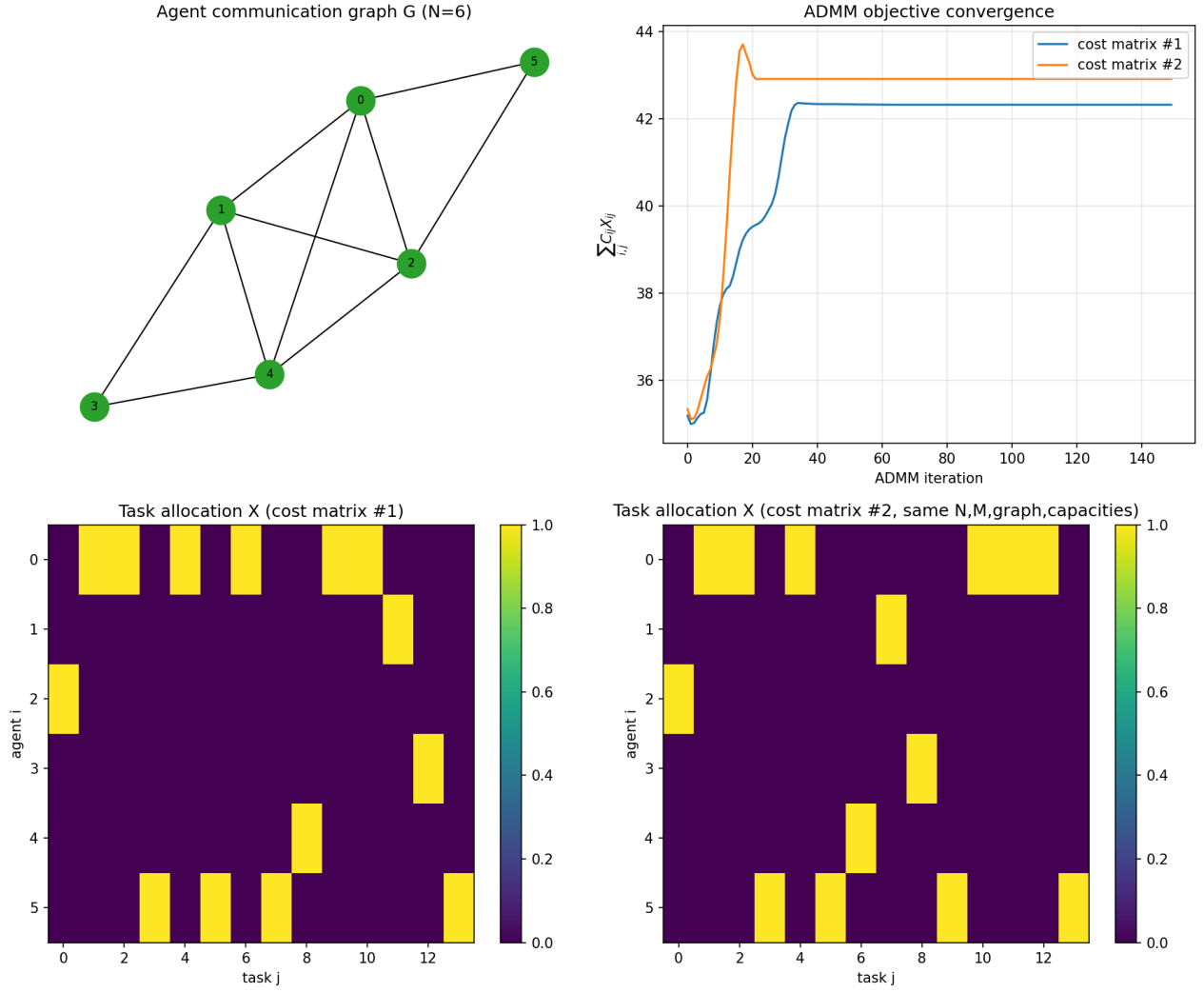


Figure 4: Top-left: communication graph, $N = 6$. Top-right: ADMM objective convergence for two different cost matrices with the same (N, M) , graph, capacities. Bottom: resulting allocation matrices X for cost matrix #1 (left) and #2 (right).

```

14 while not nx.is_connected(g):
15     trial += 1
16     g = nx.erdos_renyi_graph(n, p, seed=trial)
17 return g
18
19
20 def metropolis_weights(G):
21     n = G.number_of_nodes()
22     A = nx.to_numpy_array(G)
23     deg = A.sum(axis=1)
24     W = np.zeros((n, n))
25     for i in range(n):
26         for j in range(n):
27             if A[i, j] == 1:
28                 W[i, j] = 1.0 / (1 + max(deg[i], deg[j]))
29     W[i, i] = 1 - W[i].sum()
30 return W
31
32
33 def average_consensus(V, W, iters=200):
34     """Distributed average of the rows of V (one row per agent) using W over G."""
35     v = V.copy()
36     for _ in range(iters):
37         v = W @ v
38     return v # every row ~ column-wise average of V (to numerical precision)
39

```

```

40
41 def project_box_capacity_simplex(a, b):
42     """Euclidean projection of  $a$  in  $\mathbb{R}^M$  onto  $\{x: \sum_i x_i = b, 0 \leq x_i \leq 1\}$ .
43     Standard bisection on the KKT threshold (works since  $0 \leq b \leq M$ ). """
44     M = len(a)
45     lo, hi = a.min() - 1, a.max()
46     for _ in range(100):
47         mid = 0.5 * (lo + hi)
48         x = np.clip(a - mid, 0, 1)
49         s = x.sum()
50         if s > b:
51             lo = mid
52         else:
53             hi = mid
54     x = np.clip(a - 0.5 * (lo + hi), 0, 1)
55     return x
56
57
58 def run_instance(N, M, C, b, graph_seed=0, rho=1.0, admm_iters=150, consensus_iters=80):
59     assert M > N
60     assert b.sum() == M and (b >= 0).all()
61
62     G = connected_er(N, min(0.6, 4.0 / N + 0.2), graph_seed)
63     W = metropolis_weights(G)
64
65     x = np.zeros((N, M))
66     for i in range(N):
67         x[i] = b[i] / M # feasible-ish init
68     z = x.copy()
69     u = np.zeros((N, M))
70
71     obj_hist = []
72     for k in range(admm_iters):
73         # ---- x update: local projection for every agent (parallel / independent)
74         for i in range(N):
75             a = z[i] - u[i] - C[i] / rho
76             x[i] = project_box_capacity_simplex(a, b[i])
77
78         # ---- z update: needs sum_i (x_i + u_i); realize the sum via distributed
79         # average consensus over the graph G (only neighbour communication)
80         v = x + u
81         v_avg = average_consensus(v, W, iters=consensus_iters) # ~ (1/N) sum_i v_i, replicated at every node
82         z = v + (1.0 - N * v_avg) / N
83
84         # ---- dual update
85         u = u + x - z
86
87         obj_hist.append(float(np.sum(C * x)))
88
89     X = x.copy()
90     # column sums should be ~1 (task done once), row sums should be ~b_i
91     col_err = np.abs(X.sum(axis=0) - 1).max()
92     row_err = np.abs(X.sum(axis=1) - b).max()
93     return G, b, X, obj_hist, col_err, row_err
94
95
96 if __name__ == "__main__":
97     N, M = 6, 14
98     # fix (N, M), the graph, and the capacities b_i once; only the cost matrix C varies below
99     b = rng.integers(1, max(2, 2 * M // N), size=N)
100     b = (b / b.sum() * M).round().astype(int)
101     b[np.argmax(b)] += M - b.sum() # fix rounding so sum(b) == M exactly
102     assert b.sum() == M and (b >= 0).all()
103
104     C1 = rng.uniform(1, 10, size=(N, M))
105     G, _, X1, obj1, col_err1, row_err1 = run_instance(N, M, C1, b, graph_seed=11)
106     print(f"Cost matrix  $C_1$   $N={N}$ ,  $M={M}$ , capacities  $b={b}$ ")
107     print(f"max column sum - 1 = {col_err1:.4e} (task-once feasibility)")
108     print(f"max row sum -  $b_i$  = {row_err1:.4e} (capacity feasibility)")
109     print(f"ADMM objective (final) = {obj1[-1]:.4f}")
110
111     # hard assignment by rounding to nearest feasible (greedy on X) for a report:
112     assign1 = X1.argmax(axis=0)
113     print(f"final task->agent assignment (argmax of X):", assign1)
114

```

```

115 # second cost matrix, SAME (N,M,graph,capacities) -- only C differs
116 C2 = rng.uniform(1, 10, size=(N, M))
117 G2, _, X2, obj2, col_err2, row_err2 = run_instance(N, M, C2, b, graph_seed=11)
118 print(f"\n[Cost_matrix_{#2}]_capacities_b={b}")
119 print(f"_max_|column_sum_|_1_|_={col_err2:.4e}")
120 print(f"_max_|row_sum_|_b_|_i_|_={row_err2:.4e}")
121 print(f"_ADMM_objective_(final)_=_{obj2[-1]:.4f}")
122 assign2 = X2.argmax(axis=0)
123 print(f"_final_task->agent_assignment_(argmax_of_X):", assign2)
124 print(f"_#tasks_reassigned_to_a_different_agent_vs_cost_matrix_{#1}:",
125       int(np.sum(assign1 != assign2)))
126
127 # plots
128 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
129
130 nx.draw(G, nx.spring_layout(G, seed=1), ax=axes[0, 0], with_labels=True,
131        node_color="tab:green", node_size=400, font_size=8)
132 axes[0, 0].set_title(f"Agent_communication_graph_{G}(N={N})")
133
134 axes[0, 1].plot(obj1, label="cost_matrix_{#1}")
135 axes[0, 1].plot(obj2, label="cost_matrix_{#2}")
136 axes[0, 1].set_xlabel("ADMM_iteration")
137 axes[0, 1].set_ylabel(r"$\sum_{i,j} C_{ij} X_{ij}$")
138 axes[0, 1].set_title("ADMM_objective_convergence")
139 axes[0, 1].legend()
140 axes[0, 1].grid(alpha=0.3)
141
142 im1 = axes[1, 0].imshow(X1, aspect="auto", cmap="viridis")
143 axes[1, 0].set_title("Task_allocation_X_(cost_matrix_{#1})")
144 axes[1, 0].set_xlabel("task_{j}")
145 axes[1, 0].set_ylabel("agent_{i}")
146 fig.colorbar(im1, ax=axes[1, 0])
147
148 im2 = axes[1, 1].imshow(X2, aspect="auto", cmap="viridis")
149 axes[1, 1].set_title("Task_allocation_X_(cost_matrix_{#2},same_N,M,graph,capacities)")
150 axes[1, 1].set_xlabel("task_{j}")
151 axes[1, 1].set_ylabel("agent_{i}")
152 fig.colorbar(im2, ax=axes[1, 1])
153
154 fig.tight_layout()
155 fig.savefig("problem3_results.png", dpi=150)
156 print("\nSaved_figure_to_problem3_results.png")

```

Problem 4: LASSO via ADMM

Problem 4

Consider $\min_x \left[\frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1 \right]$. Write the ADMM iteration updates, with closed-form solutions for the x - and z -updates.

Solution:

Introduce a copy z of x and split the non-smooth ℓ_1 term onto it:

$$\min_{x,z} \frac{1}{2} \|Ax - b\|^2 + \lambda \|z\|_1 \quad \text{s.t.} \quad x - z = 0.$$

The scaled-form augmented Lagrangian (dual variable u , penalty $\rho > 0$) is

$$L_\rho(x, z, u) = \frac{1}{2} \|Ax - b\|^2 + \lambda \|z\|_1 + \frac{\rho}{2} \|x - z + u\|^2 - \frac{\rho}{2} \|u\|^2.$$

x -update. Minimizing L_ρ over x is an unconstrained quadratic:

$$x^{k+1} = \arg \min_x \frac{1}{2} \|Ax - b\|^2 + \frac{\rho}{2} \|x - z^k + u^k\|^2.$$

Setting the gradient to zero, $A^\top(Ax - b) + \rho(x - z^k + u^k) = 0$, gives the closed form

$$x^{k+1} = (A^\top A + \rho I)^{-1} (A^\top b + \rho(z^k - u^k)).$$

($A^\top A + \rho I$ is fixed across iterations, so its Cholesky/inverse is computed once.)

z -update. Minimizing L_ρ over z is a proximal operator of the (separable, scalar) ℓ_1 norm:

$$z^{k+1} = \arg \min_z \lambda \|z\|_1 + \frac{\rho}{2} \|z - (x^{k+1} + u^k)\|^2 = \mathcal{S}_{\lambda/\rho}(x^{k+1} + u^k),$$

where $\mathcal{S}_\kappa$ is the elementwise soft-thresholding operator,

$$\mathcal{S}_\kappa(a)_i = \text{sign}(a_i) \max(|a_i| - \kappa, 0).$$

Dual update.

$$u^{k+1} = u^k + x^{k+1} - z^{k+1}.$$

Numerical check. On a random sparse-recovery instance ($m = 50$ measurements, $n = 100$ unknowns, 8 true nonzeros, $\lambda = 0.3$), ADMM converges to primal residual $\|x - z\| \approx 2 \times 10^{-4}$ within 500 iterations, recovers all 8 true nonzero support indices exactly, and its final objective (2.8225) matches a general-purpose `scipy.optimize` (L-BFGS-B) solve of the same non-smooth objective (2.8234) to within numerical tolerance – confirming the derivation above is correct.

Code (`lasso_admm.py`)

```

1 import numpy as np
2
3 def soft_threshold(a, kappa):
4     return np.sign(a) * np.maximum(np.abs(a) - kappa, 0.0)
5
6 def lasso_admm(A, b, lam, rho=1.0, n_iters=300, tol=1e-8):
7     n = A.shape[1]
8     x = np.zeros(n)
9     z = np.zeros(n)
10    u = np.zeros(n)
11
12    AtA = A.T @ A
13    Atb = A.T @ b
14    lhs = AtA + rho * np.eye(n)
15    lhs_inv = np.linalg.inv(lhs) # precompute factorization once (constant across iters)
16
17    history = []
18    for k in range(n_iters):
19        x = lhs_inv @ (Atb + rho * (z - u))
20        z_new = soft_threshold(x + u, lam / rho)
21        u = u + x - z_new
22        primal_res = np.linalg.norm(x - z_new)
23        z = z_new
24        obj = 0.5 * np.linalg.norm(A @ x - b) ** 2 + lam * np.linalg.norm(x, 1)
25        history.append(obj)
26        if primal_res < tol:
27            break
28    return x, z, history
29
30 if __name__ == "__main__":
31     rng = np.random.default_rng(0)
32     m, n = 50, 100 # under-determined -> sparse recovery setting
33     A = rng.standard_normal((m, n))
34     x_true = np.zeros(n)
35     idx = rng.choice(n, size=8, replace=False)
36     x_true[idx] = rng.uniform(-3, 3, size=8)
37     b = A @ x_true + 0.01 * rng.standard_normal(m)
38
39     lam = 0.3
40     x_hat, z_hat, hist = lasso_admm(A, b, lam, rho=1.0, n_iters=500)
41
42     print("ADMM converged in", len(hist), "iterations")
43     print("final objective:", hist[-1])
44     print("||x_hat - z_hat|| (should be ~0, i.e. consensus reached):", np.linalg.norm(x_hat - z_hat))
45     print("nonzeros recovered:", np.sum(np.abs(x_hat) > 1e-3), "true nonzeros:", len(idx))
46     print("support recall (of the true nonzero indices found):",
47           len(set(np.where(np.abs(x_hat) > 1e-3)[0]) & set(idx)), "/", len(idx))

```

```

48
49 # cross-check against scipy's general-purpose convex solver on the same problem
50 try:
51     from scipy.optimize import minimize
52
53     def obj(x):
54         return 0.5 * np.sum((A @ x - b) ** 2) + lam * np.sum(np.abs(x))
55
56     res = minimize(obj, np.zeros(n), method="L-BFGS-B", options={"maxiter": 2000})
57     print("\nADMM objective:", hist[-1])
58     print("L-BFGS-B objective (smoothed reference, subgradient method, sanity check):", res.fun)
59 except Exception as e:
60     print("scipy cross-check skipped:", e)

```
